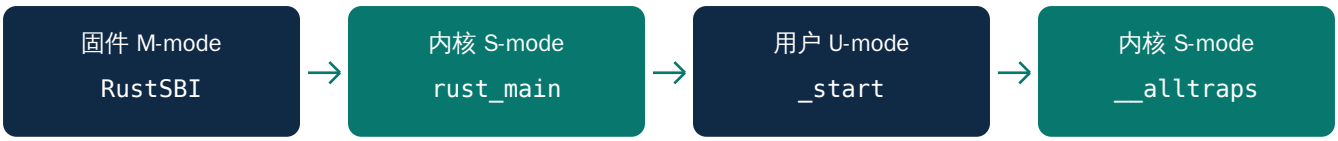
用户程序如何跑起来

构建、特权级与 Trap 源码详解

从 user/src/bin 中的 Rust 源文件追到第一次 sret，
再跟踪一次 ecall 如何进内核、改现场、返回用户态。

2026-11-11 / 工作区快照，基于 b39c63c 及当时的本地改动



目录

页	阅读路线
02–03	QEMU 启动与地址布局 · 用户镜像如何嵌入内核
04–05	新增一个用户 App · batch.rs 如何装载与换程序
06–07	TrapContext 内存布局 · CPU 如何切换特权级
08–09	trap.S 逐段拆解 · 入口保存与出口恢复
10–11	跟踪一次 write · 退出、异常与下一个 App
12–13	RISC-V 指令与汇编指令速查 · 复现实验
14–16	Rust 构建脚本与内核初始化
17–22	Rust 应用管理器、装载、栈、TrapContext 与异常分支
23–27	Rust 用户库、系统调用、日志、关机与逐函数追踪

文中的地址以一次 QEMU release 运行和 ELF 符号表为例。链接布局变化后，符号地址会变化。

从 QEMU 到 rust_main

构建工具决定镜像如何生成，固件负责进入内核，内核入口负责建立 Rust 代码运行所需的栈和全局状态。

Makefile 调用关系

```
make run
  -> make -C os run
  -> build user ELFs and .bin files
  -> build krunix ELF and krunix.bin
  -> qemu-system-riscv64 -machine virt -nographic ...
```

所在层	地址或入口	实际工作
QEMU	loader @ 0x80200000	把 krunix.bin 放到指定内存地址。-bios 指定 RustSBI 固件；-nographic 把串口输出带到终端。
固件 M-mode	RustSBI @ 0x80000000	本次固件日志记录 Supervisor Address 为 0x80200000，随后内核以 S-mode 开始执行。
内核 S-mode	_start @ 0x80200000	entry.asm 的 .text.entry 排在内核 .text 开头。la 装入 boot_stack_top，call 跳到 rust_main。

os/src/arch/riscv64/entry.asm

```
.section .text.entry
.globl _start
_start:
    la sp, boot_stack_top
    call rust_main
```

rust_main 的调用顺序

① clear_bss 清零 [sbss, ebss)。链接脚本先放 .bss.stack，再定义 sbss，清零范围避开了正在使用的启动栈。② console::init_logger 注册 logger。③ trap::init 把 stvec 配到 __alltraps。④ print_init_info 读取布局及 stvec。⑤ batch::init 输出应用表。⑥ batch::run_next_app 开始第一个应用。

内核有两个不同的栈概念。entry.asm 的启动栈是 64 KiB，用于从 _start 进入初始化；batch.rs 中 8 KiB 的 KERNEL_STACK 用于用户 Trap。另有一块 8 KiB 的 USER_STACK 给用户程序使用。

源码 Makefile:13–14 · os/Makefile:9–35 · os/src/arch/riscv64/entry.asm:1–12 · os/src/arch/riscv64/linker.ld · os/src/main.rs:25–33,102–109

用户 App 如何嵌入内核

一个 App 经历三种形态：独立链接的 ELF、提取后的裸二进制、内核只读区中保存的字节。

阶段	相关代码	产物与约定
编译	user/Makefile:7,14–18	枚举 user/src/bin/*.rs，Cargo 用 --bins 构建 release。llvm-objcopy --strip-all -O binary 把各 ELF 转成 .bin。
链接地址	user/build.rs 与 user 链接脚本	独立 App 按 0x80400000 链接。ENTRY(_start) 指定 ELF 入口，KEEP(*(.text.entry)) 把用户库入口放到代码开头。
生成表	os/build.rs:18–62	扫描同一批 .rs，按路径排序。根据 KRUNIX_APP_BIN_DIR 找 .bin，生成 OUT_DIR/link_app.S；缺文件时会报错。
嵌入	os/src/main.rs:23 与内核链接脚本	global_asm! 编译生成的汇编，.incbin 插入原始字节，KEEP(*(.rodata.apps)) 将表与 App 保留在内核 .rodata。

实际生成的 link_app.S 结构，文件路径缩写

```
.section .rodata.apps,"a"
.balign 8
.global _num_app
_num_app:
    .quad 3
    .quad app_0_start
    .quad app_1_start
    .quad app_2_start
    .quad apps_end
app_0_start:
    .incbin ".../00print.bin"
# app_1_start, app_2_start, apps_end follow
```

RV64 上 .quad 每项 8 字节。三个 App 对应一个计数和四个边界地址，共 40 字节。样例中 _num_app 是 0x80205000，所以紧接表的 app_0_start 是 0x80205028。App i 的源字节范围是 [app_start[i], app_start[i+1])，最后一项 apps_end 让最后一个 App 也能用同一公式求长度。

内核在启动前已带齐这些字节。应用运行时不解析 ELF，也不从磁盘读取；它按表找到字节，再拷贝到 App 的链接地址。

源码 user/Makefile · user/build.rs · user/src/arch/riscv64/linker.ld · os/build.rs:18–62 · os/src/main.rs:22–23

添加一个用户 App

当前构建按文件名发现 App。参考 00print，新建 user/src/bin/03hello.rs 即可加入这个批次。

示例 user/src/bin/03hello.rs

```
#![no_std]
#![no_main]

use user_lib::{self as _, println};

#[unsafe(no_mangle)]
fn main() -> i32 {
    println!("Hello from 03hello");
    0
}
```

no_std 表示使用 core 和用户库；no_main 让用户库的 _start 接管入口。use 中的 self as _ 引用 user_lib，println 宏走用户库的 write 封装。main() 的签名和 user/src/lib.rs 中 extern "Rust" 声明一致；no_mangle 保留 main 符号，使 _start 能找到它。

在项目根目录运行

```
make user
LOG=trace make run
```

make user 可以单独验证应用能编译为 ELF 与 .bin。make run 本身也会先构建用户应用，再重建内核。按路径字典序排列，03hello 接在 02power 后面，编号为 3。重命名会改变排序或编号。

入口与大小的约定

所有应用共享同一链接地址 0x80400000。用户链接脚本要求 ebss 不超过该地址加 128 KiB，覆盖代码、静态数据与 BSS 的布局；当前 AppManager 最多记录 16 个应用，边界数组多留一个槽给 apps_end。

每个 App 的 _start 先按用户链接符号清 BSS，再执行 exit(main())。main 返回 0 会发 exit(0)，内核随后转去装载下一项。

实际验证

在隔离副本中添加上述文件，QEMU 输出 Number of apps: 4、Loading app 3 和 Hello from 03hello。四个程序用完后仍复现该批次的边界问题，报 Invalid app_id: 4。

源码 user/src/bin/00print.rs · user/src/lib.rs:14–35 · user/src/arch/riscv64/linker.ld:33 · os/src/batch.rs:10,52–56

batch.rs 如何选择和装载 App

AppManager 持有嵌入镜像的地址表。所有应用按次序复用一块执行区与一对栈。

读取应用表

APP_MANAGER 是 lazy_static。batch::init 调用 print_app_info 时首次访问它，取 _num_app 的链接地址，转成 *const usize，用 read_volatile 读取计数。num_app_ptr.add(1) 跳过计数，借出 num_app + 1 个边界值，再拷入 app_start。current_app 从 0 开始，记录下一次要装载的编号。

load_app 的地址与动作

```
source = [app_start[id], app_start[id + 1])
size   = app_start[id + 1] - app_start[id]
clear  = [0x80400000, 0x80420000)
copy source -> [0x80400000, 0x80400000 + size)
fence.i
```

先清执行区，再拷贝选中的裸镜像，避免旧应用在这个区间留下的数据。以 00print 为例，本次源地址为 0x80205028，长度 3880 字节；运行入口仍为 0x80400000。两处地址一处存档、一处执行。

fence.i 使当前 hart 后续的取指看到它此前的写入。拷贝是数据写入，下一步却要从同一地址取指，因此这里要做取指同步。在多 hart 场景，单条本地 fence.i 不会同步其他 hart。

启动下一项的顺序

run_next_app 的控制流

```
exclusive_access() -> get_current_app() -> load_app(id)
-> move_to_next_app() -> drop(app_manager)
-> app_init_context(0x80400000, USER_STACK.get_sp())
-> KERNEL_STACK.push_context(cx) -> __restore(cx_addr)
```

UPSafeCell 以 RefCell 实现可变借用；它依赖当前单核使用约定。drop 显式释放借用很重要，__restore 最终执行 sret，旧 Rust 调用栈不会像普通函数返回一样执行析构。之后若 exit 又借用 APP_MANAGER，前一次借用必须已释放。

源码 os/src/batch.rs:71–156 · os/src/sync/up.rs:3–18 · os/src/syscall/fs.rs:17–20

TrapContext 是栈上的 272 字节

Rust 与汇编通过固定偏移共享同一块内存。首次启动时由 Rust 构造；运行中发生 Trap 时由汇编写入。

槽位	相对起点	对应数据
0	0x000	x[0] 保留槽。硬件 x0 恒为零，汇编无需保存。
1	0x008	x[1] / ra，应用的返回地址。
2	0x010	x[2] / sp，应用栈指针；通过 sscratch 保存与恢复。
3	0x018	x[3] / gp，全局指针。
4	0x020	x[4] / tp，线程指针。当前汇编跳过它。
5–31	0x028–0x0F8	通用寄存器。a0 在 0x050，a7 在 0x088。
32	0x100	sstatus，保存特权级和中断相关状态。
33	0x108	sepc，sret 的返回 PC。

TrapContext 用 #[repr(C)] 固定字段顺序。当前 RV64 中 usize 和 Sstatus 各占 8 字节，x[32] 共 256 字节，再加两个 8 字节字段，合计 34 × 8 = 272（0x110）。272 是 16 的倍数，所以从对齐的内核栈顶减去它，调用 handler 时仍满足整数 ABI 的 16 字节栈对齐。

首次启动时的初始现场

context.rs 与 batch.rs 对照

```
sstatus = read sstatus; set SPP = User
x = [0; 32]; x[2] = USER_STACK.get_sp()
sepc = 0x80400000
cx_addr = KERNEL_STACK.get_sp() - size_of::<TrapContext>()
```

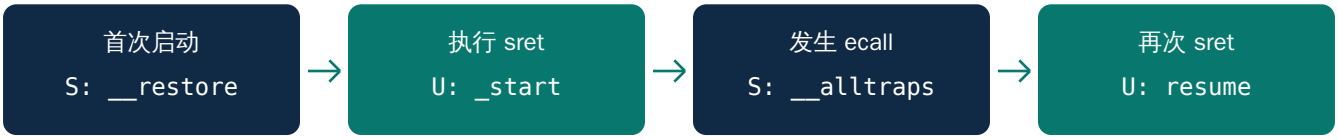
get_sp() 返回数组尾后一字节的地址作为空栈顶。栈向低地址增长，push_context 在顶部下方写入现场。x 数组初始化为零，但 __restore 会跳过 tp，硬件 x4 仍保留之前的值。

运行中再次入 Trap 时，保存和恢复使用同一偏移。例如 trap_handler 写 cx.x[10]，__restore 的 ld a0, 0x50(sp) 就会把新值交还给用户程序。

源码 os/src/trap/context.rs:1–30 · os/src/batch.rs:32–50,149–153 · os/src/trap/trap.S:16–35,43–64

硬件何时改变特权级

同一 hart 可以运行 M、S、U 三种权限的代码。本项目的 M-mode 是 RustSBI，S-mode 是 krunix，U-mode 是各个用户程序。



CSR	本项目里的作用
stvec (0x105)	保存 S-mode Trap 的基地址和最低两位模式。Direct = 0，所有送达 S-mode 的 Trap 都从同一地址开始；Vectored 对中断按编号取 BASE + 4 × cause，异常仍从 BASE 开始。
sstatus (0x100)	SPP（bit 8）记来源特权级，U=0、S=1。SPIE（bit 5）保存原 SIE，SIE（bit 1）控制 S-mode 的全局中断使能。
sepc (0x141)	进入 S-mode Trap 时保存相关指令地址，返回时供 sret 设置 PC。
scause (0x142)	最高位区分中断和异常，其余位记录原因编号。用户 ecall 的同步异常编号是 8。
stval (0x143)	补充信息随异常类型和平台而异，访存异常可记录出错地址。
sscratch (0x140)	软件约定的临时寄存器，用户运行期间存内核栈顶。硬件进 Trap 时不会替内核换栈。

U → S 时 CPU 做什么

以已委托给 S-mode 的用户 ecall 为例，CPU 写 sepc、scause 等 Trap CSR，把原特权级记入 SPP，把 SIE 复制到 SPIE，再清 SIE，进入 S-mode，并从 stvec 的 BASE 取指。通用寄存器继续保持原值，保存现场由入口汇编完成。

异常委托由 M-mode 配置。medeleg 的第 8 位控制 U-mode ecall 是否委托给 S-mode；本次运行确实进入了 S-mode handler。内核用 SBI 输出字符时发起的是 S-mode ecall，进入的是固件服务，和应用的 write syscall 是两层接口。

S → U 时 sret 做什么

sret 从 SPP 决定返回权限，从 sepc 取目标 PC，并把 SPIE 恢复到 SIE、置 SPIE 为 1、把 SPP 清为 0。首次启动时 Rust 主动设 SPP=User；普通 Trap 返回时用硬件记录后由汇编保存的状态。

源码 os/src/trap/mod.rs:14–24 · context.rs:18–28。硬件语义见第 13 页 RISC-V Privileged ISA 与 Zicsr。

逐段读 __alltraps

全局汇编通过 global_asm!(include_str!("trap.S")) 进入内核。trap::init 使用 riscv crate 写 stvec，入口地址来自链接器符号。

os/src/trap/trap.S:12–38，省略注释

```
__alltraps:
    csrrw sp, sscratch, sp
    addi sp, sp, -34*8
    sd x1, 1*8(sp)
    sd x3, 3*8(sp)
    .set n, 5
    .rept 27
        SAVE_GP %n
        .set n, n+1
    .endr
    csrr t0, sstatus
    csrr t1, sepc
    sd t0, 32*8(sp)
    sd t1, 33*8(sp)
    csrr t2, sscratch
    sd t2, 2*8(sp)
    mv a0, sp
    call trap_handler
```

① csrrw 同时把旧 sscratch 放入 sp，把旧 sp 写进 sscratch。应用栈指针暂存在 CSR，CPU 开始使用内核栈。② addi 下移 sp 预留现场；先保存 ra、gp、x5–x31。x0 不可变，x4 按应用约定跳过。

③ .altmacro 允许 %n 把当前汇编符号 n 求值后传给宏。宏中的反斜线 n 用实参替换，SAVE_GP %n 第一次展开为 sd x5, 5*8(sp)。.rept 27 在汇编阶段生成 x5 至 x31 的保存指令。

④ t0–t2 已保存在现场，可以用它们读取 sstatus、sepc 和 sscratch。x[2] 存的是进入 Trap 前的用户 sp。⑤ mv a0, sp 传递 &mut TrapContext；call 设置 ra 并进入 Rust，返回点恰好是 __restore。

文件开头的保存与恢复宏

```
.altmacro
.macro SAVE_GP n
    sd x\n, \n*8(sp)
.endm
.macro LOAD_GP n
    ld x\n, \n*8(sp)
.endm
```

源码 os/src/trap/mod.rs:12–27 · os/src/trap/trap.S:1–40。宏求值见第 13 页 GNU as 文档。

逐段读 __restore

首次启动由 run_next_app 直接传入新现场。正常 Trap 处理完后，从 trap_handler 返回到同一段汇编，a0 带回要恢复的现场地址。

os/src/trap/trap.S:40–64，省略注释

```
__restore:
    mv sp, a0
    ld t0, 32*8(sp)
    ld t1, 33*8(sp)
    ld t2, 2*8(sp)
    csrw sstatus, t0
    csrw sepc, t1
    csrw sscratch, t2
    ld x1, 1*8(sp)
    ld x3, 3*8(sp)
    .set n, 5
    .rept 27
        LOAD_GP %n
        .set n, n+1
    .endr
    addi sp, sp, 34*8
    csrrw sp, sscratch, sp
    sret
```

执行点	sp	sscratch
mv sp, a0 之后	现场起点	原值。首次进入时不能假设它是用户 sp。
csrw sscratch, t2 之后	现场起点	从 x[2] 取出的用户 sp
addi sp, sp, 272 之后	内核栈顶	用户 sp
csrrw sp, sscratch, sp 之后	用户 sp	内核栈顶，留给下一次 Trap

先写 CSR，后恢复 x5–x31，可避免临时寄存器 t0–t2 的用户值被覆盖。恢复 x10 后 a0 已经是应用的值，此时现场地址仍由 sp 保持。最后 sret 依据 sstatus 和 sepc 切换权限、设置 PC。

当前 trap_handler 有 #[unsafe(no_mangle)]，实际链接符号已核对，实测 a0 传参及返回可用。函数仍采用 Rust 默认 ABI；若要把汇编接口写成稳定的调用约定，可显式声明 extern "C"。

源码 os/src/trap/trap.S:40–64 · os/src/trap/mod.rs:26–27,65–66 · os/src/batch.rs:145–155

跟踪 00print 的一次 write

用户态格式化输出会经过 krunix 的 syscall；内核真正向终端输出字符时，再借助 RustSBI 的控制台服务。

从用户字符串到 QEMU 串口

```
00print::main -> println! -> user_lib::console::print
-> Stdout::write_str -> user_lib::write -> sys_write
-> ecall -> __alltraps -> trap_handler -> kernel sys_write
-> kernel console::print -> sbi::console_putchar
```

寄存器	发起 write 时	恢复后
a7 / x17	64，krunix 的 write 号	从现场恢复
a0 / x10	第一个参数 fd=1	返回写入长度 len
a1 / x11	第二个参数 buf.as_ptr()	从现场恢复
a2 / x12	第三个参数 buf.len()	从现场恢复

user/src/syscall.rs:9–15

```
asm!(
    "ecall",
    inlateout("x10") args[0] => ret,
    in("x11") args[1],
    in("x12") args[2],
    in("x17") id
);
```

inlateout 指定同一个寄存器先承载输入、后承载输出。trap_handler 读 scause 和 stval，将原因转换为 riscv::interrupt::Trap。UserEnvCall 分支从 x[17] 和 x[10..=12] 取编号及三个参数。sys_write 用 fd=1 分支读用户缓冲、解码 UTF-8、打印并返回 len。

handler 把 cx.sepc 加 4，因为 ecall 是固定 32 位指令，sepc 原先指向 ecall 本身。handler 再把返回值写入 cx.x[10]。__restore 恢复 a0 并执行 sret，用户内联汇编读到 ret，继续执行 ecall 后的指令。

普通 Rust 函数调用和 ecall 是不同的接口。call 通过 ra 保存返回地址、权限不变；ecall 触发同步异常、由硬件进入更高权限，返回地址由 sepc 管理。这里的 64、93 是 krunix 选用的系统调用号。

源码 user/src/{console.rs,lib.rs,syscall.rs} · os/src/trap/mod.rs:26–43 · os/src/syscall/{mod.rs,fs.rs} · os/src/{console.rs,sbi.rs}

退出、异常与下一个 App

用实际的三个 App 对照处理器和批处理器的分工。正常 write 会恢复原 App，退出与这两类应用异常会直接启动下一项。

应用	触发原因	内核路径
00print	write(64)；main 返回 0 后 exit(93)	write 返回原现场。exit 记录退出码，进入 run_next_app，构造下一项的新现场。
01store_fault	向地址 0 写字节，产生 StoreFault 或 StorePageFault 分支能处理的异常	handler 打印 stval 与 cx.sepc，进入 run_next_app。日志统称 Page fault，当前分支同时接受 access fault 和 page fault。
02power	循环计算 3 的幂模 10007，多次 write 后 exit(0)	每次 write 恢复原程序；最后 exit 试图装载编号 3，触发越界 panic。

QEMU 实测输出摘录

```
[INFO] Number of apps: 3
[Kernel] Loading app 0 at address 0x80205028
Hello World
[kernel] Application exited with code 0
[Kernel] Loading app 1 at address 0x80205f50
[kernel] Page fault in application, bad addr = 0x0, ...
[Kernel] Loading app 2 at address 0x80206ef0
Test power OK!
[kernel] Application exited with code 0
Panicked at os/src/batch.rs:73 Invalid app_id: 3
```

为什么 exit 不回到原应用

sys_exit 直接调用返回类型为 ! 的 run_next_app。下一项覆盖固定执行区，重用内核栈顶写入新的 TrapContext，再由 __restore 用 sret 跳到新应用的 _start。发生存储异常或非法指令时也走这条路径。

这版代码的边界

这里是单核顺序批处理。当前路径没有按应用配置页表或独立地址空间，sys_write 直接解释用户缓冲区指针。入口默认 Trap 来自 U-mode，未区分来自 S-mode 的嵌套 Trap；也未保存浮点、向量和 tp 现场。未知 Trap 与未知 syscall 走 panic。

最后一次 exit 时 current_app 等于 num_app，load_app 的边界检查会 panic。panic handler 通过 SBI 以故障原因关机，本次 make run 的最终退出码为 2。

源码 user/src/bin/*.rs · os/src/trap/mod.rs:39–63 · os/src/batch.rs:71–74,136–156 · os/src/lang_items.rs:4–16

RISC-V 指令与汇编指示符

寄存器别名和汇编指示符出现在源码里，机器真正执行的是链接后的指令编码。

写法	语义及此处用途
la sp, symbol	伪指令。取符号地址，供 _start 设置启动栈。具体展开由汇编器、链接和寻址方式决定。
addi rd, rs, imm	rd ← rs + 带符号立即数。±272 在 12 位有符号范围内；用于分配和释放现场。
sd rs, imm(sp) / ld rd, imm(sp)	RV64 以 sp + imm 为字节地址写入或读出 8 字节。每个寄存器槽位因此步长为 8。
mv rd, rs	伪指令，整数语义相当于 addi rd, rs, 0。源码中用来在 a0 与 sp 之间复制地址。
call symbol / ret	伪指令。call 记录返回地址 ra，常见展开是 auipc ra 与 jalr；ret 通常用 jalr 跳回 ra。链接器可放松编码。
csrr rd, csr	伪指令，等价于 csrrs rd, csr, x0，只读取 CSR。
csrw csr, rs	伪指令，等价于 csrrw x0, csr, rs，写 CSR 并丢弃旧值。
csrrw rd, csr, rs	读取 CSR 旧值到 rd，同时写入 rs 的旧值；rd 与 rs 都写 sp 时完成交换。
ecall / sret	ecall 请求环境服务，触发同步异常；sret 从 S-mode 按 SPP、sepc 返回。
fence.i	把当前 hart 的先前数据写入与之后取指同步。load_app 拷贝代码后执行。

汇编指示符	汇编或链接阶段的作用
.section / .globl / .align	选择段、导出符号、在 RISC-V 中按 2 的幂对齐。入口的 .align 2 表示 4 字节对齐。
.altmacro / .macro / .set / .rept	定义展开规则和汇编期变量，重复生成 save/load 指令。
.space / .balign / .quad / .incbin	预留字节、按给定字节数对齐、写 8 字节表项、嵌入二进制文件。

本次反汇编中，addi、sd、ld、mv 有 16 位压缩编码。程序仍按源码中的寄存器和字节偏移工作。ecall 与 sret 是 32 位指令，处理 ecall 时 sepc += 4 与其他指令是否压缩无关。

本次 call 的机器码分成两条。auipc ra, 0x1 把当前 PC 加上 0x1000 写入 ra；jalr 0x324(ra) 用该基址加 0x324 得到目标 0x80201388，同时把下一条指令地址 0x8020006c 写进 ra。那条下一指令就是 __restore 的开头。

常用别名 x0=zero · x1=ra · x2=sp · x3=gp · x4=tp · x5–x7=t0–t2 · x10–x17=a0–a7。s0–s11 是 ABI 规定的被调用者保存寄存器，t0–t6 是临时寄存器。Trap 入口主动保存两组，以便用户现场在返回时完整恢复。

对照自己的 ELF 和源码

以下命令在 krunix 项目根目录运行。保留 ELF 可看符号和反汇编，.bin 则用于 QEMU 加载。

构建、符号、反汇编和生成文件

```
LOG=trace make run
dir=target/riscv64gc-unknown-none-elf/release
llvm-nm -n "$dir/krunix" | rg 'trap_handler|__alltraps|__restore'
llvm-nm -n "$dir/00print" | rg ' (_start|main)$'
llvm-objdump -d --disassemble-symbols=__alltraps,__restore \
    "$dir/krunix"
find "$dir/build" -name link_app.S
```

本次 ELF 符号	地址	核对内容
内核 _start	0x80200000	固件传入的内核入口
__alltraps / __restore	0x80200010 / 0x8020006c	stvec 的基地址 / handler 普通返回后接续处
trap_handler	0x80201388	本次 call 反汇编目标
_num_app / app_0_start	0x80205000 / 0x80205028	应用表起点 / 第一个源镜像字节
00print::_start / main	0x80400000 / 0x8040005c	sret 跳到用户库入口，再调用用户 main

源码入口

构建读根 Makefile、os/Makefile、user/Makefile 与两侧 build.rs；布局读内核和用户的 arch/riscv64/linker.ld。运行顺序从 os/src/main.rs 到 batch.rs，再读 os/src/trap/{mod.rs,context.rs,trap.S}，系统调用接着读 user/src/syscall.rs 与 os/src/syscall/{mod.rs,fs.rs}。

当前活跃入口来自 os/src/trap/trap.S。仓库另有 os/src/arch/riscv64/trap.asm，包含 user_trap_entry 与 enter_user，当前构建入口没有包含该文件。本报告的反汇编与运行验证针对 __alltraps 和 __restore。

规范与工具文档

RISC-V Privileged ISA · Supervisor-Level ISA（stvec、sstatus、sepc、sret）

RISC-V Unprivileged ISA · Zicsr 与 Zifencei

RISC-V ELF psABI · 整数寄存器、传参与栈对齐

GNU as · .altmacro 与汇编期表达式求值

验证 release 内核 QEMU 实跑、llvm-nm 与 llvm-objdump 对照。新增 03hello 在隔离副本中实测。

Rust 构建脚本如何驱动链接

Cargo 在主机上运行 build.rs。脚本打印 cargo:: 指令，把参数传给 Cargo，再由 Cargo 编译目标程序。

user/build.rs:7–11

```
 7      println!("cargo::rerun-if-changed={}", linker_script.display());
 8      println!(
 9          "cargo::rustc-link-arg-bins=-T{}",
10          manifest_dir.join(linker_script).display()
11      );
```

os/build.rs:9–28

```
 9  fn main() {
10      let linker_script = Path::new("src/arch/riscv64/linker.ld");
11      let manifest_dir = Path::new(env!("CARGO_MANIFEST_DIR"));
12      println!("cargo::rerun-if-changed={}", linker_script.display());
13      println!(
14          "cargo::rustc-link-arg-bin=krunix=-T{}",
15          manifest_dir.join(linker_script).display()
16      );
17
18      let source_dir = manifest_dir.join("../user/src/bin");
19      println!("cargo::rerun-if-changed={}", source_dir.display());
20      println!("cargo::rerun-if-env-changed=KRUNIX_APP_BIN_DIR");
21      let bin_dir = env::var_os("KRUNIX_APP_BIN_DIR")
22          .map(PathBuf::from)
23          .unwrap_or_else(|| {
24              manifest_dir
25                  .join("../target")
26                  .join(env::var("TARGET").unwrap())
27                  .join("release")
28          });
```

代码	作用和边界
CARGO_MANIFEST_DIR	编译脚本时取得当前包的绝对目录，用 join 把链接脚本拼成绝对路径。即使从仓库根目录构建，-T 仍能定位正确文件。
rustc-link-arg-bins / bin=krunix	前者把 -T 传给该包的各个用户二进制，后者只传给 krunix。链接脚本指定各自的基址与段布局。
rerun-if-changed / rerun-if-env-changed	告诉 Cargo 跟踪脚本依赖。源码目录、单个 .bin、链接脚本、KRUNIX_APP_BIN_DIR 的变化会触发对应的重新生成。
var_os(...).map(PathBuf::from)	把可选环境值转成路径，保留 OS 字符串形式。Makefile 显式传 release 目录；直接构建内核时回退到 ../target/<TARGET>/release。

内核有 release 和 dev 两种构建入口，用户 App 始终先按 release 生成。build.rs 负责收集与嵌入已经生成的裸镜像，构建用户镜像的是 user/Makefile。

生成应用地址表的每一步

这个脚本把文件名变成有顺序的应用编号，再把字节内容交给汇编器和链接器。

os/build.rs:30–43

```
30      let mut apps: Vec<_> = fs::read_dir(source_dir)
31          .unwrap()
32          .map(|entry| entry.unwrap().path())
33          .filter(|path| path.extension() == Some(OsStr::new("rs")))
34          .collect();
35      apps.sort();
36
37      let mut asm =
38          String::from(".section .rodata.apps,\"a\"\n.balign 8\n.global
            _num_app\n_num_app:\n");
39      writeln!(asm, "    .quad {}", apps.len()).unwrap();
40      for index in 0..apps.len() {
41          writeln!(asm, "    .quad app_{index}_start").unwrap();
42      }
43      asm.push_str("    .quad apps_end\n");
```

os/build.rs:45–62

```
45      for (index, source) in apps.iter().enumerate() {
46          let binary = bin_dir
47              .join(source.file_name().unwrap())
48              .with_extension("bin");
49          println!("cargo::rerun-if-changed={}", binary.display());
50          assert!(
51              binary.is_file(),
52              "missing {}; run make user first",
53              binary.display()
54          );
55          writeln!(asm, "app_{index}_start:\n    .incbin {:?}", binary).unwrap();
56      }
57      asm.push_str("apps_end:\n");
58      fs::write(
59          PathBuf::from(env::var_os("OUT_DIR").unwrap()).join("link_app.S"),
60          asm,
61      )
62      .unwrap();
```

30–35 行枚举直接子项，取路径，只保留扩展名为 rs 的项，排序后 index 由 0 开始。这个顺序与编号前缀一致，例如 00print、01store_fault、02power。这里的 Vec 与 String 是主机构建时使用的 std 类型。

37–43 行先建立只读段和全局符号，再生成数量、每个起始标签、最后的结束标签。writeln! 向 String 追加文本，符号的数值由链接器填入 .quad。

45–62 行把每个源文件的文件名改成 .bin，去 bin_dir 找镜像；is_file 检查失败就终止构建。.incbin 把镜像字节原样写入段，apps_end 给最后一个镜像定界，fs::write 将整份汇编写到 OUT_DIR。

main.rs 的 Rust 初始化

启动汇编把控制权交给 rust_main 后，Rust 才开始访问全局状态、配置 Trap 和选择第一个 App。

os/src/main.rs:22–33

```
22  global_asm!(include_str!("arch/riscv64/entry.asm"));
23  global_asm!(include_str!(concat!(env!("OUT_DIR"), "/link_app.S")));
24
25  #[unsafe(no_mangle)]
26  pub extern "C" fn rust_main() {
27      clear_bss();
28      console::init_logger();
29      trap::init();
30      print_init_info();
31      batch::init();
32      batch::run_next_app();
33  }
```

os/src/main.rs:102–109

```
102  fn clear_bss() {
103      unsafe extern "C" {
104          safe fn sbss();
105          safe fn ebss();
106      }
107      (linker_symbol_addr!(sbss)..linker_symbol_addr!(ebss))
108          .for_each(|a| unsafe { (a as *mut u8).write_volatile(0) });
109  }
```

代码	为什么需要它
#![no_std] / #![no_main]	内核依赖 core 而无宿主标准库；自己的 _start 由链接脚本和全局汇编提供。
linker_symbol_addr!	把符号转换为指针再提取数值地址。sbss、ebss、_num_app 等在这里是地址标记。写成 extern 函数声明是为了引用符号，代码从不调用这些标记。
global_asm! + include_str!	编译时读取汇编文本。entry.asm 提供 _start；生成的 link_app.S 提供 App 表和镜像。concat! 把 OUT_DIR 与文件名连成路径。
#[unsafe(no_mangle)] / extern C	前者保留 rust_main 符号，后者明确调用约定，因此 entry.asm 的 call rust_main 有可链接、可调用的目标。
clear_bss / write_volatile	在 sbss..ebss 上迭代，对每个地址调用 (a as *mut u8).write_volatile(0)。区间必须有效可写。volatile 保留写入动作，没有线程同步语义。

先清 BSS，再启动 logger 与配置 stvec。print_init_info 读取包版本、链接符号范围和 stvec 的实际值，段大小用 end - start 计算，包含链接脚本在符号边界之间的对齐空间。batch::init 读取应用表并打印；run_next_app 之后不再沿 rust_main 返回。

AppManager 怎样读出嵌入表

编译期生成的 .quad 序列在运行时被解释为 RV64 usize 数组。这个解释依赖两侧布局约定一致。

os/src/batch.rs:52–56

```
52  struct AppManager {
53      num_app: usize,
54      current_app: usize,
55      app_start: [usize; MAX_APP_NUM + 1],
56  }
```

os/src/batch.rs:107–125

```
107  lazy_static! {
108      static ref APP_MANAGER: UPSafeCell<AppManager> = unsafe {
109          UPSafeCell::new({
110              unsafe extern "C" {
111                  safe fn _num_app();
112              }
113              let num_app_ptr = linker_symbol_addr!(_num_app) as *const usize;
114              let num_app = num_app_ptr.read_volatile();
115              let mut app_start: [usize; MAX_APP_NUM + 1] = [0; MAX_APP_NUM + 1];
116              let app_start_raw: &[usize] =
117                  core::slice::from_raw_parts(num_app_ptr.add(1), num_app + 1);
118              app_start[..=num_app].copy_from_slice(app_start_raw);
119              AppManager {
120                  num_app,
121                  current_app: 0,
122                  app_start,
123              }
124          })
125      };
```

num_app 是数量，current_app 是下一次装载编号。app_start 有 17 个槽，可保存最多 16 个 App 的起点以及最后的终点。首次 exclusive_access() 会触发 lazy_static 初始化。

read_volatile 读取 _num_app 地址处的 8 字节。num_app_ptr.add(1) 按 usize 步长加 8，指向第一个镜像边界。from_raw_parts 借出 N+1 项，copy_from_slice 把它们写进固定数组的 0..=N。源表对齐来自 .balign 8，长度来自生成脚本。

print_app_info 用相邻边界相减得到镜像大小。若表中的数量超过 MAX_APP_NUM，当前实现会在数组切片时失败；生成脚本本身没有检查这个数量。

load_app 的指针、切片和取指同步

应用的存档在内核只读区，实际执行区是另一个地址。load_app 做编号检查、清零、复制和取指同步。

os/src/batch.rs:71–96

```
71      unsafe fn load_app(&self, app_id: usize) {
72          if app_id >= self.num_app {
73              panic!("Invalid app_id: {}", app_id);
74          }
75          println!(
76              "[Kernel] Loading app {} at address {:#x}",
77              app_id, self.app_start[app_id]
78          );
79
80          unsafe {
81              core::slice::from_raw_parts_mut(APP_BASE_ADDRESS as *mut u8,
            APP_SIZE_LIMIT).fill(0);
82          }
83          let app_src = unsafe {
84              core::slice::from_raw_parts(
85                  self.app_start[app_id] as *const u8,
86                  self.app_start[app_id + 1] - self.app_start[app_id],
87              )
88          };
89
90          let app_dst =
91              unsafe { core::slice::from_raw_parts_mut(APP_BASE_ADDRESS as *mut u8,
            app_src.len()) };
92          app_dst.copy_from_slice(app_src);
93          unsafe {
94              asm!("fence.i");
95          }
96      }
```

代码	执行效果
71–78 行	拒绝 app_id ≥ num_app。Loading app 日志中的地址是源镜像地址，因此每个 App 不同。
80–82 行	从 0x80400000 借出 128 KiB 的可变字节切片，fill(0) 清掉上一个镜像留在执行区里的内容。
83–91 行	从地址表构造只读源切片；再借出目标地址处同长度的可变切片。指针加长度必须对应可访问的内存。
92–95 行	copy_from_slice 要求两个切片等长，源与目标不可重叠；fence.i 保证当前 hart 后续取指能看到新指令。

APP_SIZE_LIMIT 用于清零长度，目标切片的长度来自 app_src.len()。内核此处没有再比较两者；正常构建依赖用户链接脚本约束镜像布局及生成表的正确性。

以 00print 为例，读取 [0x80205028, 0x80205f50)，写到 [0x80400000, 0x80400f28)。接下来的 PC 会从 0x80400000 起跑。

两块栈、可变借用与非返回跳转

run_next_app 连接 AppManager 与 __restore。它在切换栈之前把运行下一项需要的 Rust 状态准备好。

os/src/batch.rs:37–43

```
37      fn push_context(&self, cx: TrapContext) -> &'static mut TrapContext {
38          let cx_ptr = (self.get_sp() - core::mem::size_of::<TrapContext>()) as *mut
            TrapContext;
39          unsafe {
40              *cx_ptr = cx;
41          }
42          unsafe { cx_ptr.as_mut().unwrap() }
43      }
```

os/src/batch.rs:136–156

```
136  pub fn run_next_app() -> ! {
137      let mut app_manager = APP_MANAGER.exclusive_access();
138      let current_app = app_manager.get_current_app();
139      unsafe {
140          app_manager.load_app(current_app);
141      }
142      app_manager.move_to_next_app();
143      drop(app_manager);
144
145      unsafe extern "C" {
146          fn __restore(cx_addr: usize);
147      }
148
149      unsafe {
150          __restore(KERNEL_STACK.push_context(TrapContext::app_init_context(
151              APP_BASE_ADDRESS,
152              USER_STACK.get_sp(),
153          )) as *const _ as usize);
154      }
155      panic!("Unreachable in batch::run_current_app!");
156  }
```

两块静态栈各 8 KiB，结构体按 4096 对齐。get_sp() 取字节数组末端作为空栈顶。push_context 把 272 字节的现场写在内核栈顶下方，用地址转裸指针，再将其解释为可变引用。

UPSafeCell::exclusive_access() 用 RefCell::borrow_mut() 返回 RefMut，运行时检查独占借用。unsafe impl Sync 允许静态共享，依赖当前单核且无并发重入的使用约定。先递增 current_app 再 drop 守卫，否则换栈后旧调用链不会执行析构。

extern C 声明把现场地址送入 a0。Rust 签名里的 __restore 没有写成返回！，所以尾部 panic 用来兜底；正常路径由 sret 进入 U-mode。push_context 的裸指针写入和 &'static mut 依赖栈独占及生命周期约束。普通共享静态字节数组并不自动提供 Rust 的内部可变性。

另见 os/src/batch.rs:8–30 · os/src/sync/up.rs:3–18 · 第 09 页汇编恢复路径

构造 TrapContext 和安装 Trap 向量

这里有两种完全不同的写入。app_init_context 修改内存里的待恢复状态，trap::init 修改正在运行的 hart 的 CSR。

os/src/trap/context.rs:3–11

```
 3  #[repr(C)]
 4  pub struct TrapContext {
 5      // 0x00
 6      pub x: [usize; 32],
 7      // 0x100
 8      pub sstatus: Sstatus,
 9      // 0x108
10      pub sepc: usize,
11  }
```

os/src/trap/context.rs:18–29

```
18      pub fn app_init_context(entry: usize, sp: usize) -> Self {
19          let mut sstatus = sstatus::read();
20          sstatus.set_spp(SPP::User);
21          let cx = TrapContext {
22              x: [0; 32],
23              sstatus,
24              sepc: entry,
25          };
26          let mut cx = cx;
27          cx.set_sp(sp);
28          cx
29      }
```

os/src/trap/mod.rs:12–24

```
12  global_asm!(include_str!("trap.S"));
13
14  pub fn init() {
15      unsafe extern "C" {
16          safe fn __alltraps();
17      }
18      unsafe {
19          stvec::write(stvec::Stvec::new(
20              linker_symbol_addr!(__alltraps),
21              stvec::TrapMode::Direct,
22          ));
23      }
24  }
```

repr(C) 让字段按声明顺序布局。set_sp 写 x[2]。sstatus::read 取当前 CSR 的快照，set_spp(SPP::User) 只改快照的 SPP。sepc 字段用传入的 entry，最终由 __restore 写回两个 CSR，然后 sret 生效。

trap::init 中 Stvec::new 构造地址加 Direct 模式，要求地址 4 字节对齐；stvec::write 才写硬件。这个入口的前提是可执行的汇编、正确的栈约定，以及后续能返回的现场。安装向量本身不会调用 __alltraps。

trap_handler 如何读原因和返回现场

硬件把原因写入 CSR，汇编把通用寄存器与返回状态交给 Rust。两类数据在 handler 中合起来决定下一步。

os/src/trap/mod.rs:26–43

```
26  #[unsafe(no_mangle)]
27  pub fn trap_handler(cx: &mut TrapContext) -> &mut TrapContext {
28      let scause = scause::read();
29      let stval = stval::read();
30      let trap: Trap<Interrupt, Exception> = match scause.cause().try_into() {
31          Ok(trap) => trap,
32          Err(_) => panic!(
33              "Unsupported trap {:?}, stval = {:#x}!",
34              scause.cause(),
35              stval
36          ),
37      };
38
39      match trap {
40          Trap::Exception(Exception::UserEnvCall) => {
41              cx.sepc += 4;
42              cx.x[10] = syscall(cx.x[17], [cx.x[10], cx.x[11], cx.x[12]]) as usize;
43          }
```

a0 的四个时刻

```
a0 on Rust entry  = address of TrapContext
cx.x[10]          = saved user a0
a0 on Rust return = address returned by trap_handler
a0 after ld       = modified cx.x[10]
```

阶段	具体读写
解码	scause::read 取快照；cause() 区分异常和中断；try_into 转为带类型的 Trap<Interrupt, Exception>。转换失败直接 panic。
匹配 ecall	只有 UserEnvCall 执行 syscall。其他异常和中断由后续 match 分支决定；能解码不代表已有处理逻辑。
更新返回点	cx.sepc += 4 改内存，CSR 在 __restore 才被重写。若是 exit，syscall 不返回旧现场。
写返回值	x[17] 是号，x[10]、x[11]、x[12] 是前三个参数。isize 返回值转为 usize 保存寄存器位模式，之后用户端以 isize 读取。
跨汇编边界	no_mangle 保留 call 目标名；当前函数仍是 Rust ABI。本次编译实测指针在 a0 传入并返回。正常结束时最后一行 cx 把同一个现场交给 __restore。

用户 a0 与 Rust 入口 a0 同名但出现于不同时段。__alltraps 先把用户 a0 保存在 x[10]，再用硬件 a0 传现场地址，因此用户的 fd 等参数仍能读出来。

异常分支与批次边界

一条异常分支如果调用 run_next_app，恢复的是新应用的初始现场。普通 write 才恢复当前 App 保存下来的现场。

os/src/trap/mod.rs:44–66

```
44          Trap::Exception(Exception::StoreFault) | Trap::Exception(Exception::StorePageFault)
            => {
45              println!(
46                  "[kernel] Page fault in application, bad addr = {:#x}, bad instruction =
            {:#x}, core dumped!",
47                  stval, cx.sepc
48              );
49              run_next_app();
50          }
51          Trap::Exception(Exception::IllegalInstruction) => {
52              println!(
53                  "[kernel] Illegal instruction in application, bad instruction = {:#x}, core
            dumped!",
54                  cx.sepc
55              );
56              run_next_app();
57          }
58          _ => panic!(
59              "[kernel] Unsupported trap {:?}, stval = {:#x}, core dumped!",
60              scause.cause(),
61              stval
62          ),
63      }
64
65      cx
66  }
```

匹配分支	发生什么
StoreFault | StorePageFault	模式中的 | 匹配两类存储异常。stval 给出出错地址信息，cx.sepc 是触发异常的 PC。日志文字写 Page fault，即便原因是 access fault。
IllegalInstruction	报告触发异常的指令地址后进入下一个 App。代码打印的是 sepc 地址，而非从该地址读出的机器码。
_ / 转换失败	未知或未处理原因进入 panic 路径。字符串里的 core dumped 是日志措辞，仓库这条路径没有写转储文件。

程序正常 exit 和两个异常分支共用 run_next_app。举例，运行 app 1 时 current_app 已是 2；异常发生就装载 app 2。app 2 exit 时 current_app 已是 3，load_app(3) 在边界检查中 panic。

入口汇编固定执行 sp 与 sscratch 交换。处理用户 Trap 期间 sscratch 暂存用户栈指针，若内核此时再触发 S-mode Trap，当前入口无法用这一约定安全处理嵌套。

源码 os/src/trap/mod.rs:29–66 · os/src/batch.rs:71–74,136–156 · os/src/lang_items.rs

user/src/lib.rs 怎样调用用户 main

真正的用户入口在公共库里。每个 App 文件只定义自己的 main，链接阶段把它与公共入口配成一个镜像。

user/src/lib.rs:14–43

```
14  unsafe extern "Rust" {
15      fn main() -> i32;
16  }
17
18  #[unsafe(no_mangle)]
19  #[unsafe(link_section = ".text.entry")]
20  extern "C" fn _start() -> ! {
21      clear_bss();
22      unsafe {
23          exit(main());
24      }
25      panic!("unreachable after sys_exit!");
26  }
27
28  fn clear_bss() {
29      unsafe extern "C" {
30          safe fn sbss();
31          safe fn ebss();
32      }
33      (linker_symbol_addr!(sbss)..linker_symbol_addr!(ebss))
34          .for_each(|a| unsafe { (a as *mut u8).write_volatile(0) });
35  }
36
37  pub fn write(fd: usize, buf: &[u8]) -> isize {
38      syscall::sys_write(fd, buf)
39  }
40
41  pub fn exit(xstate: i32) -> isize {
42      syscall::sys_exit(xstate)
43  }
```

user/src/bin/00print.rs:6–10

```
 6  #[unsafe(no_mangle)]
 7  fn main() -> i32 {
 8      println!("Hello World");
 9      0
10  }
```

unsafe extern Rust 声明外部 main() -> i32，00print.rs 通过 no_mangle 提供它。用户库的 _start 用 link_section 放进 .text.entry，链接脚本把该段放在最前面，因此 0x80400000 落到 _start。

内核清过执行区；用户 _start 仍按自己的 sbss/ebss 再清一次 BSS，以链接布局定义静态数据的初始化范围。随后先求 main() 的 i32 返回值，再交给 exit。_start 返回！，因为正常 exit 交给内核切换应用。

write 与 exit 是供 App 调用的薄封装。用户的 panic handler 打印文件、行号和消息，随后执行 unreachable!()；当前实现没有显式发 exit。

另见 user/src/arch/riscv64/linker.ld:1–33 · user/src/lang_items.rs:4–15

从 println! 到内联 ecall

user/src/console.rs 负责 Rust 的格式化，user/src/syscall.rs 负责把参数放进约定的寄存器。

user/src/console.rs:9–18

```
 9  impl Write for Stdout {
10      fn write_str(&mut self, s: &str) -> core::fmt::Result {
11          write(STDOUT, s.as_bytes());
12          Ok(())
13      }
14  }
15
16  pub fn print(args: fmt::Arguments) {
17      Stdout.write_fmt(args).unwrap();
18  }
```

user/src/console.rs:27–32

```
27  #[macro_export]
28  macro_rules! println {
29      ($fmt: literal $(, $($arg: tt)+)?) => {
30          $crate::console::print(format_args!(concat!($fmt, "\n") $(, $($arg)+)?));
31      }
32  }
```

user/src/syscall.rs:6–26

```
 6  fn syscall(id: usize, args: [usize; 3]) -> isize {
 7      let mut ret: isize;
 8      unsafe {
 9          asm!(
10              "ecall",
11              inlateout("x10") args[0]=> ret,
12              in("x11") args[1],
13              in("x12") args[2],
14              in("x17") id
15          );
16      }
17      ret
18  }
19
20  pub fn sys_write(fd: usize, buf: &[u8]) -> isize {
21      syscall(SYSCALL_WRITE, [fd, buf.as_ptr() as usize, buf.len()])
22  }
23
24  pub fn sys_exit(xstate: i32) -> isize {
25      syscall(SYSCALL_EXIT, [xstate as usize, 0, 0])
26  }
```

println! 用 concat! 补换行，format_args! 产生 fmt::Arguments，$crate 绑定宏所属的 user_lib。Stdout 实现 core::fmt::Write，所以 write_fmt 可以按片段调用 write_str；一次 println! 可能产生多次 write。

write_str 将 &str 转为 &[u8]。sys_write 取指针和字节数并连同 fd=1 打包。内联汇编用 in 指定输入，用 inlateout 让 x10 既接收参数又回传结果。sys_exit 把 i32 退出码放入第一个槽，另外两个槽置零。

syscall 分发、写输出和退出

内核接到的是寄存器中的整数。syscall 用编号解释这些整数，再把它们交给不同的 Rust 函数。

os/src/syscall/mod.rs:7–16

```
 7  const SYSCALL_EXIT: usize = 93;
 8  const SYSCALL_WRITE: usize = 64;
 9
10  pub fn syscall(id: usize, args: [usize; 3]) -> isize {
11      match id {
12          SYSCALL_WRITE => sys_write(args[0], args[1] as *const u8, args[2]),
13          SYSCALL_EXIT => sys_exit(args[0] as i32),
14          _ => panic!("Unsupported syscall id = {}", id),
15      }
16  }
```

os/src/syscall/fs.rs:3–20

```
 3  const FD_STDOUT: usize = 1;
 4
 5  pub fn sys_write(fd: usize, buf: *const u8, len: usize) -> isize {
 6      match fd {
 7          FD_STDOUT => {
 8              let slice = unsafe { core::slice::from_raw_parts(buf, len) };
 9              let str = core::str::from_utf8(slice).unwrap();
10              print!("{}", str);
11              len as isize
12          }
13          _ => panic!("Unsupported file descriptor = {}", fd),
14      }
15  }
16
17  pub fn sys_exit(xstate: i32) -> isize {
18      println!("[kernel] Application exited with code {}", xstate);
19      crate::batch::run_next_app()
20  }
```

编号或操作	代码里的数据流
write = 64	args[0] 是 fd，args[1] 转成 *const u8，args[2] 是字节数。当前只处理 FD_STDOUT = 1。
from_raw_parts / from_utf8	构造借用的字节切片需要缓冲地址、长度和生命周期有效。这里直接信任指针；UTF-8 解码失败时 unwrap 会 panic。
print! / len as isize	将字符串交给内核 console，返回字节长度。handler 把它保存到用户 a0 的槽位。
exit = 93	args[0] 转回 i32，打印退出码，随后 run_next_app() 的 ! 能适配函数声明的 isize 返回类型。旧应用不再继续。

sys_write 的 print! 是内核宏，输出走 SBI，不会再次调用用户态 write。未知 fd 或 syscall 编号会 panic。当前裸地址实现没有用户缓冲区验证，这个约定只适合当前受信任的实验 App。

内核日志最终怎样到终端

内核的普通 print! 与 log::info! 在同一个内核控制台汇合。日志级别只过滤走 log crate 的记录。

os/src/console.rs:6–17

```
 6  impl Write for Stdout {
 7      fn write_str(&mut self, s: &str) -> fmt::Result {
 8          for c in s.chars() {
 9              console_putchar(c as usize);
10          }
11          Ok(())
12      }
13  }
14
15  pub fn print(args: fmt::Arguments) {
16      Stdout.write_fmt(args).unwrap();
17  }
```

os/src/console.rs:46–55

```
46  pub fn init_logger() {
47      static LOGGER: ConsoleLogger = ConsoleLogger;
48
49      let level = option_env!("LOG")
50          .unwrap_or("info")
51          .parse()
52          .unwrap_or(log::LevelFilter::Off);
53      log::set_logger(&LOGGER).unwrap();
54      log::set_max_level(level);
55  }
```

代码	运行时表现
init_logger / option_env!	在编译时读取 LOG，未设置用 info，不能解析时用 Off。set_logger 注册静态 ConsoleLogger，set_max_level 安装级别过滤。
ConsoleLogger::log	enabled 比较记录级别与上限。log 对不同级别选 ANSI 颜色，输出级别名和 record.args()，再重置颜色。flush 为空。
print / write_fmt / write_str	将格式化参数写入 Stdout，随后逐个 Rust char 交给 SBI legacy console_putchar。当前样例使用 ASCII；这里没有显式按 UTF-8 字节编码输出。
panic handler / shutdown	打印 PanicInfo 的位置和消息，然后 shutdown(true) 发 SBI System Reset，原因是 SystemFailure。batch 用完的 panic 因此以失败退出 QEMU。

LOG=trace make run 可以放开所有日志级别。Hello World、Loading app 与 panic 走普通打印，所以 LOG=off 仍然可见。RustSBI 的启动横幅也不经过内核 logger。

源码 os/src/console.rs:19–69 · os/src/lang_items.rs:4–17 · os/src/sbi.rs:3–15

逐函数跟踪三个示例

这张对照表把 Rust 的函数调用、应用索引、Trap 恢复目标连起来。前面的源码页可用来逐项定位。

时刻	Rust 路径	现场与结果
初次运行	rust_main → batch::init → run_next_app	读取计数 3，选 app 0，再把 current_app 加到 1。构造入口 0x80400000 的现场。
00print 输出	user::_start → main → println! → write → trap_handler → sys_write	ecall 把调用点存入 sepc；Rust 在内存里把 sepc 加 4，将 len 放到 x[10]，恢复 app 0。
00print 退出	main 返回 0 → user::exit → sys_exit → run_next_app	current_app 从 1 变 2。覆盖 App 执行区，新的现场指向 app 1。
01store_fault 异常	main 用 write_volatile 往地址 0 写入 → trap_handler → run_next_app	发生访存异常，当前 handler 记录 stval 和故障 PC；current_app 从 2 变 3，启动 app 2。
02power 计算	main 循环更新数组，定期 println!；随后 main 返回 0	pow[index] 由上一次结果乘 3 后模 10007 得出。每 10000 步输出一次，最后打印 Test power OK!。
批次结束	exit → run_next_app → load_app(3) → panic handler → shutdown(true)	3 已等于 num_app。没有新的 TrapContext 被安装，固件收到故障关机请求。

Rust 文件与章节索引

代码范围	定位页
user/build.rs、os/build.rs	14–15
os/src/main.rs、console.rs、sbi.rs、lang_items.rs	16、26
os/src/batch.rs、sync/up.rs	17–19
os/src/trap/context.rs、trap/mod.rs、trap/trap.S	20–22、08–09
user/src/lib.rs、console.rs、syscall.rs、lang_items.rs	23–24
os/src/syscall/mod.rs、syscall/fs.rs、三个 App	25、27

路径按工作区当前构建入口展开。新增 03hello 的步骤和独立验证结果见第 04 页。